OBSW Developer Guide

8 FreeRTOS Tasks + main.c — Complete Reference

7 Tasks MTQ-Only	6-Stage Boot Sequence	Build Fixed -std=gnu11	Sim Running Verified

WHAT IS IN THIS GUIDE?

This guide covers all 7 FreeRTOS simulation tasks and the main.c boot sequence for SLARS-BCG-1. It documents every file with complete working code, the exact errors encountered during development, how each was diagnosed and fixed, and the complete build and run procedure. MTQ-Only revision: reaction wheel monitor task removed.

7	6	3	GNU11	100ms
FreeRTOS Tasks	Boot Stages	Build Errors Fixed	Compiler Flag	Sim Loop Tick

Table of Contents

1. Architecture Overview — 7 Tasks 3
2. Repository Structure 3
3. Environment Setup 4
4. slars_tasks.h — Shared Header 4
5. watchdog_task.c — Priority 8 5
6. adcs_controller.c — Priority 7 6
7. fdir_monitor.c — Priority 6 8
8. power_manager.c — Priority 6 9
9. comms_manager.c — Priority 5 10
10. payload_scheduler.c — Priority 5 11
11. telemetry_task.c — Priority 4 12
12. flash_manager.c — Priority 3 13
13. main.c — 6-Stage Boot + Loop 14
14. Updated Makefile 16
15. Errors Encountered and Fixes 17
16. Build and Run — Complete Steps 19
17. Expected Simulation Output 19
18. What Comes Next — Step D EKF 20

1. Architecture Overview — 7 FreeRTOS Tasks

SLARS-BCG-1 runs 7 FreeRTOS tasks (down from 8 — the RW monitor task was removed in the MTQ-Only revision). Each task owns one subsystem. In simulation all tasks run in a single-threaded loop. On the real STM32 OBC, FreeRTOS runs them concurrently using xTaskCreate() + vTaskStartScheduler().

Task File	Priority	Period	Subsystem	Key Function
watchdog_task.c	P8	100ms	Watchdog	Kicks HW WDT, monitors task deadlines
adcs_controller.c	P7	100ms	ADCS MTQ	B-dot, PD, PID — 3-axis MTQ only
fdir_monitor.c	P6	2000ms	FDIR	4-level fault escalation, no RW check
power_manager.c	P6	1000ms	Power/EPS	Battery rules, science gate 8 deg
comms_manager.c	P5	500ms	COMMS	UHF beacon via tm_encoder, S-band CFDP

payload_scheduler.c	P5	1000ms	Payload	SGP4, Sri Lanka gate, 4 conditions
telemetry_task.c	P4	10000ms	Telemetry	HK TM frame, MTQ dipoles, daily reset
flash_manager.c	P3	5000ms	Flash/eMMC	LittleFS, 90% purge, wear levelling

NOTE: *The old codebase had 8 tasks including an RW monitor. That task is removed in the MTQ-Only revision — there is no reaction wheel.*

2. Repository Structure

Directory tree:

```
~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/

■■■ Makefile <- -std=gnu11 for usleep support

■■■ src/

■ ■■■ main.c <- 6-stage boot + simulation loop

■ ■■■ hal/

■ ■ ■■■ sim_hal.h <- extern declarations ONLY (inside #ifndef)

■ ■ ■■■ sim_state.c <- ALL global definitions (compiled once)

■ ■ ■■■ slars_tasks.h <- shared includes + function declarations

■ ■■■ tasks/

■ ■ ■■■ watchdog_task.c

■ ■ ■■■ adcs_controller.c

■ ■ ■■■ fdir_monitor.c

■ ■ ■■■ power_manager.c

■ ■ ■■■ comms_manager.c

■ ■ ■■■ payload_scheduler.c

■ ■ ■■■ telemetry_task.c

■ ■ ■■■ flash_manager.c

■ ■■■ tc/ <- Step A: TC dispatcher
```

```
■ ■■■ tm/ <- Step B: TM encoder

■ ■■■ adcs/ <- Step D: EKF (next)

■■■ tests/

■ ■■■ test_tc.c

■ ■■■ test_tm.c

■■■ ground/ <- Step C: Python ground station
```

3. Environment Setup

Terminal commands:

```
# Install gcc and build tools (Ubuntu/Kali/Debian)

sudo apt update && sudo apt install gcc build-essential make git -y


# Create all folders

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/hal

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tc

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tm

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/adcs

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/tests

mkdir -p ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/ground
```

4. src/hal/slars_tasks.h — Shared Header

Every task file includes slars_tasks.h instead of sim_hal.h directly. This header pulls in all system includes (stdio, stdlib, math, time, unistd) and declares all cross-task function signatures so any task can call any other task's public functions.
nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/hal/slars_tasks.h

```
#ifndef SLARS_TASKS_H

#define SLARS_TASKS_H

#include "sim_hal.h"

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <math.h>

#include <time.h>

#include <stdarg.h> #include

<unistd.h> /* usleep() */


void watchdog_task(void);

void watchdog_task_kick(uint8_t task_id);

void adcs_controller_task(void);

void fdir_monitor_task(void);

void fdir_log(const char *fmt, ...);

void fdir_report_task_dead(uint8_t task_id);
```

```
void fdir_power_cycle_subsystem(uint8_t id);

void power_manager_task(void);

void power_notify_uhf_pa(uint8_t active);

void power_notify_sband_tx(uint8_t active);

void comms_manager_task(void);

void comms_trigger_sband_downlink(void);

void payload_scheduler_task(void);

void payload_scheduler_set_max_passes(uint8_t n);

void payload_scheduler_reset_daily(void);

void telemetry_task(void);

void flash_manager_task(void);


#endif /* SLARS_TASKS_H */
```

5. src/tasks/watchdog_task.c — Priority 8 | 100ms

Highest priority task. Runs every 100ms. Each task calls watchdog_task_kick(task_id) once per cycle. If any task misses 5 consecutive cycles the watchdog reports it. On real STM32: IWDG->KR = 0xAAAA resets the hardware watchdog timer.

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/watchdog_task.c

```
#include "../hal/slars_tasks.h"

#define NUM_TASKS 7

static volatile uint8_t task_alive[NUM_TASKS] = {0};

static int wdt_cycle = 0;


void watchdog_task_kick(uint8_t task_id) {

if (task_id < NUM_TASKS) task_alive[task_id] = 1;

}


void watchdog_task(void) {

printf("[WDT] Watchdog started P8 100ms\n");

printf("[WDT] Monitoring 7 tasks (MTQ-Only).\n");

for (;;) {

wdt_cycle++; /* Real STM32: IWDG-

>KR = 0xAAAA */

for (uint8_t i = 1; i < NUM_TASKS; i++) {

if (task_alive[i] == 0 && wdt_cycle > 5)

printf("[WDT] WARNING: Task %d missed deadline!\n", i);

task_alive[i] = 0;

}

task_alive[0] = 1;

if (wdt_cycle % 100 == 0)

printf("[WDT] All tasks alive cycle %d\n", wdt_cycle);

return; /* simulation: single-shot on first call */

}

}
```

6. src/tasks/adcs_controller.c — Priority 7 | 100ms

Second highest priority. 10 Hz control loop. Implements three MTQ-Only control modes — all using sim_mtq_dipole[3], no reaction wheel. Science gate is 8 deg (was 5 deg with RW).

Mode	Control Law	Gains	When Active

1 — B-dot	m = -k * dB/dt	K_BDOT = 2e4	Post-launch until rate < 3 deg/s
2 — PD	T = -Kp*e - Kd*de/dt	KP=5e-5 KD=3e-4	Nominal orbit, rate < 3 deg/s
3 — PID	T=-Kp*e-Ki*int(e)-Kd*de/dt	KP=6e-5 KI=1.5e-6 KD=4e-4	120s before science pass

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/adcs_controller.c

```
#include "../hal/slars_tasks.h"

#include <math.h>

#define K_BDOT 2e4f #define KP_PD 5e-5f #define KD_PD 3e-4f

#define KP_PID 6e-5f #define KI_PID 1.5e-6f #define KD_PID 4e-4f

#define I_CLAMP 0.5f #define DT 0.1f #define SCIENCE_GATE_DEG

8.0f /* MTQ-Only: was 5 deg with RW */

static float integral[3]={0},prev_B[3]={0},prev_err[3]={0};

/* Mode 1: B-dot -- m = -k * dB/dt */

static void mode1_bdot(void) {

float B[3]; B[0]=sim_magnetic_B[0]+((float)(rand()%100-

50))*1e-9f; /* ... Y and Z similarly ... */

float m[3];

m[0]=-K_BDOT*(B[0]-prev_B[0])/DT; /* clamp to

0.20 A.m2 and set sim_mtq_dipole[i] */

sim_angular_rate *= 0.998f;

if (sim_angular_rate < 3.0f && sim_adcs_mode==1) {

sim_adcs_mode = 2; /* detumbling done */

}

}

/* Mode 2: PD -- 3-axis MTQ */

static void mode2_pd(void) {

float e[3] = {sim_pointing_err*0.6f, ...};

float er[3] = {sim_angular_rate*0.6f, ...};

float T[i] = -KP_PD*e[i] - KD_PD*er[i];

/* clamp and set sim_mtq_dipole[i] */

sim_pointing_err *= 0.995f; /* simulate convergence */

}

/* Mode 3: PID -- anti-windup, 3-axis MTQ */
```

```
static void mode3_pid(void) {
integral[i] += e[i]*DT;
if (integral[i] > I_CLAMP) integral[i] = I_CLAMP;
T[i] = -KP_PID*e[i] - KI_PID*integral[i] -
KD_PID*de; /* clamp and set sim_mtq_dipole[i] */ }

void adcs_controller_task(void) {
watchdog_task_kick(1);
switch (sim_adcs_mode) {
case 1: mode1_bdot(); break;
case 2: mode2_pd(); break;
case 3: mode3_pid(); break;
}
if (adcs_cycle % 50 == 0)
printf("[ADCS] Mode=%s rate=%.2f err=%.1f MTQ=[%.3f,%.3f,%.3f]\n",...);
}
```

7. src/tasks/fdir_monitor.c — Priority 6 | 2000ms

4-level fault escalation. Monitors 4 health parameters every 2s. MTQ-Only: RW saturation check removed. Each parameter must fail FAULT_THRESHOLD (3) consecutive checks before escalating.

Level	Trigger	Action
L1	Task misses WDT deadline	Restart the dead FreeRTOS task
L2	SoC below 20% twice OR ADCS rate > 15 deg/s	Power cycle subsystem
L3	Battery critical (SoC < 20% or Vbat < 6V)	Force SAFE MODE, payloads off
L4	Level 3 persists 3 orbits	Transmit anomaly report, await TC

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/fdir_monitor.c

```
#include "../hal/slars_tasks.h"

#include <stdarg.h>

#define FAULT_THRESHOLD 3

void fdir_monitor_task(void) {

watchdog_task_kick(2);

float soc = sim_soc;

float vbat = sim_battery_v;

float rate = sim_angular_rate;

float att = sim_pointing_err;

/* MTQ-Only: 4 checks (was 5 with RW saturation check) */

check(0, soc < 0.20f, 3, "Battery SoC");

check(1, vbat < 6.0f, 3, "Battery Voltage");

check(2, rate > 15.0f, 2, "ADCS Angular Rate");

check(3, att > 14.0f && sim_adcs_mode==3, 1, "ADCS Pointing Mode3");

/* NOTE: RW saturation check REMOVED -- no reaction wheel */

}
```

8. src/tasks/power_manager.c — Priority 6 | 1000ms

Guardian of the battery. Enforces every Step 2 power rule. Science gate now requires att_err < 8 deg (MTQ-Only cannot achieve better than 5 deg). Hard rule: UHF PA + S-band TX never simultaneous.

Rule	Threshold	Action
Safe entry	SoC < 20% OR Vbat < 6.0V	All payloads off, beacon only
Safe exit	SoC > 70% AND 2+ orbits in safe	Return to NOMINAL
Science gate	SoC > 60% AND att < 8 deg	Allow SCIENCE mode
EOL reduction	SoC stays below 60%	Reduce to 2 passes/day

TX sequencing	UHF PA active	Block S-band TX until PA off
Emergency	SoC < 10%	Force-disable all non-essential

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/power_manager.c

```
#define SCIENCE_ATT_GATE_DEG 8.0f /* MTQ-Only: was 5 deg with RW */

void power_manager_task(void) {
watchdog_task_kick(3);
float soc=sim_soc, vbat=sim_battery_v, att=sim_pointing_err;

if (soc < SOC_SAFE_ENTRY || vbat < VOLTAGE_CUTOFF) {
sim_mode = MODE_SAFE; /* entry */ } else if
(sim_mode==MODE_NOMINAL && soc>SOC_SCIENCE_GATE && att <
SCIENCE_ATT_GATE_DEG) {
sim_mode = MODE_SCIENCE;
}

/* HARD RULE: UHF PA + S-band TX never simultaneous */
if (uhf_pa_active && sband_tx_active) {
sband_tx_active = 0;
printf("[PWR] POWER VIOLATION BLOCKED!\n");
}
}
```

9. src/tasks/comms_manager.c — Priority 5 | 500ms

Transmits UHF beacon every 30s using tm_encode_housekeeping() from Step B. This is what connects the TM encoder to the running satellite simulation. Also handles S-band CFDP downlink after science passes. **nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/comms_manager.c**

```
#include "../hal/slars_tasks.h" #include

"../tm/tm_encoder.h" /* uses Step B TM encoder */ #define

BEACON_INTERVAL_S 30


void comms_manager_task(void) {

watchdog_task_kick(4);

uint32_t now = (uint32_t)time(NULL);


if (now - last_beacon_time >= BEACON_INTERVAL_S) {

uint8_t frame[128];

tm_encode_housekeeping(frame); /* 128-byte CSP frame */

power_notify_uhf_pa(1);

sim_beacon_count++; /* simulate UHF transmit */

power_notify_uhf_pa(0);

last_beacon_time = now;

printf("[COMMS] Beacon #%lu TX soc=%.0f%%\n",...);

}

if (sband_pending) {

power_notify_sband_tx(1);

printf("[COMMS] S-band CFDP transmitting...\n");

power_notify_sband_tx(0);

sband_pending = 0;

}

}
```

10. src/tasks/payload_scheduler.c — Priority 5 | 1000ms

Computes satellite position using SGP4 stub. All 4 science gates must pass simultaneously before imaging opens. Activates ADCS Mode 3 two minutes before the Sri Lanka pass.

Gate	Condition	Why Required
1 — Footprint	Lat 5.9N-9.9N, Lon 79.7E-81.9E	No imaging when not over Sri Lanka
2 — Battery	SoC > 60%	Step 2: science needs sufficient energy
3 — Pointing	att_err < 8 deg (MTQ-Only)	Imager needs accurate nadir pointing
4 — Pass co	untpasses_today < max_passes (3 nomin	al)Step 2: daily energy balance

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/payload_scheduler.c

```
#define SL_LAT_MIN 5.9f #define SL_LAT_MAX 9.9f

#define SL_LON_MIN 79.7f #define SL_LON_MAX 81.9f

#define SCIENCE_ATT_GATE_DEG 8.0f /* MTQ-Only */

void payload_scheduler_task(void) {

watchdog_task_kick(5);

double lat, lon;

get_position((double)time(NULL), &lat, &lon);

if (over_sri_lanka((float)lat, (float)lon)) {

uint8_t ok_soc = (sim_soc > 0.60f);

uint8_t ok_att = (sim_pointing_err < SCIENCE_ATT_GATE_DEG);

uint8_t ok_pass = (passes_today < max_passes_sched);

if (ok_soc && ok_att && ok_pass && !science_active) {

sim_payload_en[0] = true; /* imager ON */

sim_payload_en[1] = true; /* SDR relay ON */

science_active = 1; } }

else if (science_active) {

sim_payload_en[0] = sim_payload_en[1] = false;

passes_today++;

comms_trigger_sband_downlink();

sim_adcs_mode = 2; /* back to coarse pointing */

}

}
```

11. src/tasks/telemetry_task.c — Priority 4 | 10000ms

Assembles and prints the full housekeeping telemetry frame every 10s. MTQ-Only: shows sim_mtq_dipole[3] instead of rw_speed. Resets daily counters at UTC midnight.

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/telemetry_task.c

```
#include "../hal/slars_tasks.h"

#include "../tm/tm_encoder.h"


void telemetry_task(void) {

watchdog_task_kick(6);

tm_count++;

printf("[TM] Frame #%lu\n", tm_count);

printf("[TM] Bat: soc=%.0f%% vbat=%.2fV\n",

sim_soc*100, sim_battery_v);

printf("[TM] Mode: %s ADCS=%d att=%.1fdeg rate=%.2fdeg/s\n",

sim_mode==0?"SAFE":"NOMINAL", sim_adcs_mode,

sim_pointing_err, sim_angular_rate);

printf("[TM] MTQ: X=%.3f Y=%.3f Z=%.3f A.m2\n",

sim_mtq_dipole[0], sim_mtq_dipole[1], sim_mtq_dipole[2]);

/* Reset daily counters at midnight */

if (((uint32_t)time(NULL) % 86400) < 10)

payload_scheduler_reset_daily();

}
```

12. src/tasks/flash_manager.c — Priority 3 | 5000ms

Lowest priority task — only runs when nothing else needs the CPU. Monitors eMMC health, simulates data accumulation, purges at 90% full.

Partition	Size	Content	Retention Policy
Science imagery	400 MB	JPEG2000 images	Delete after CFDP confirmed
AIS vessel data	50 MB	NMEA AIS sentences	48-hour rolling window
Housekeeping log	50 MB	Every TM frame	30-day rolling window
Anomaly files	12 MB	FDIR fault data	Keep until ground request

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/tasks/flash_manager.c

```
#define SCIENCE_MAX_MB 400


void flash_manager_task(void) {

watchdog_task_kick(7);
```

```
science_mb++; hk_frames++;

float usage = (float)science_mb / SCIENCE_MAX_MB * 100.0f;

sim_flash_usage_pct = (uint8_t)usage;

if (usage > 90.0f) {

science_mb = (uint32_t)(science_mb * 0.7f); /* purge 30% */

printf("[FLASH] Storage >90%% -- purging\n");

}

}
```

13. src/main.c — 6-Stage Boot + Simulation Loop

Entry point. Runs the 6-stage boot sequence then enters the simulation loop calling tasks at their correct periods. On the real OBC, replace the loop with xTaskCreate() calls + vTaskStartScheduler().

Stage	Duration	Action	Failure Handling
1 — System init	< 500ms	HAL init, srand()	Watchdog armed immediately
2 — Task init	< 200ms	Print 7 task list	Task failure shown in output
3 — Health check	< 5s	EPS, ADCS queried	Failed subsystem flagged
4 — TLE + IGRF	< 1s	Orbit data loaded	Uses sim placeholder
5 — Mode det.	< 500ms	Read SoC, set mode	Defaults to SAFE if low
6 — Operational	forever	All tasks running	FDIR monitors continuously

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/src/main.c

```
#include "hal/slars_tasks.h"


int main(void) {

/* Stage 1 */

srand((unsigned)time(NULL));

printf("[BOOT] Stage 1: Hardware init...\n");


/* Stage 2: print 7 task list (MTQ-Only, no RW monitor) */

/* Stage 3: health check EPS, ADCS, OBC */

/* Stage 4: TLE + IGRF load */ /* Stage

5: mode determination from SoC */

sim_mode = (sim_soc < 0.20f) ? MODE_SAFE : MODE_NOMINAL;


/* Stage 6: operational */

watchdog_task(); /* print WDT banner once */
```

```
/* Simulation loop -- each cycle = 100ms */

int cycle = 0;

for (;;) {

cycle++;

adcs_controller_task(); /* every 100ms */

if (cycle%5 ==0) comms_manager_task(); /* every 500ms */

if (cycle%10 ==0) power_manager_task(); /* every 1000ms */

if (cycle%10 ==0) payload_scheduler_task();

if (cycle%20 ==0) fdir_monitor_task(); /* every 2000ms */

if (cycle%50 ==0) flash_manager_task(); /* every 5000ms */

if (cycle%100==0) telemetry_task(); /* every 10s */

/* simulate battery */

if (cycle%30==0) sim_soc -= 0.001f;

if (cycle%20==0 && sim_soc<0.95f) sim_soc += 0.002f;

usleep(100000); /* 100ms real time */

}

}


/* On real OBC replace loop with:

xTaskCreate(adcs_controller_task,"ADCS",4096,NULL,7,NULL);

xTaskCreate(fdir_monitor_task, "FDIR",2048,NULL,6,NULL);

xTaskCreate(power_manager_task, "PWR", 2048,NULL,6,NULL);

xTaskCreate(comms_manager_task, "COMM",4096,NULL,5,NULL);

xTaskCreate(payload_scheduler_task,"SCHED",4096,NULL,5,NULL);

xTaskCreate(telemetry_task, "TM", 2048,NULL,4,NULL);

xTaskCreate(flash_manager_task, "FLASH",2048,NULL,3,NULL);

vTaskStartScheduler(); */
```

14. Updated Makefile

WARNING: TABs required + -std=gnu11 is critical

Lines starting with $(CC) and rm MUST use a real TAB character. The key change from the Step A/B Makefile is -std=gnu11 (was -std=c11). This enables GNU extensions including usleep(). Without this, gcc reports "implicit declaration of usleep".

nano ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW/Makefile

```
CC = gcc CFLAGS = -Wall -Wextra -Wno-unused-parameter -std=gnu11 -O0

-g -I./src

LIBS = -lm

HAL_SRC = src/hal/sim_state.c

TASK_DIR = src/tasks

TASK_SRCS = \

$(TASK_DIR)/watchdog_task.c \

$(TASK_DIR)/adcs_controller.c \

$(TASK_DIR)/fdir_monitor.c \

$(TASK_DIR)/power_manager.c \

$(TASK_DIR)/comms_manager.c \

$(TASK_DIR)/payload_scheduler.c \

$(TASK_DIR)/telemetry_task.c \

$(TASK_DIR)/flash_manager.c


slars_obsw_sim: src/main.c src/tm/tm_encoder.c $(TASK_SRCS) $(HAL_SRC)

$(CC) $(CFLAGS) -o slars_obsw_sim \

src/main.c src/tm/tm_encoder.c $(TASK_SRCS) $(HAL_SRC) $(LIBS)


test_tc: tests/test_tc.c src/tc/tc_dispatcher.c $(HAL_SRC) $(CC) $(CFLAGS)

-o test_tc tests/test_tc.c src/tc/tc_dispatcher.c $(HAL_SRC)


test_tm: tests/test_tm.c src/tm/tm_encoder.c $(HAL_SRC) $(CC) $(CFLAGS) -o

test_tm tests/test_tm.c src/tm/tm_encoder.c $(HAL_SRC) $(LIBS)


all: slars_obsw_sim test_tc test_tm

run: slars_obsw_sim

./slars_obsw_sim

clean:

rm -f test_tc test_tm slars_obsw_sim

.PHONY: all run clean
```

15. Errors Encountered and How We Fixed Them

These are the three exact errors produced when building the tasks for the first time. Each error, its root cause, and the precise fix is documented below.

Error 1 — Variables undeclared in task files

Error messages:

```
src/tasks/adcs_controller.c: error: sim_angular_rate undeclared

src/tasks/fdir_monitor.c: error: sim_pointing_err undeclared

src/tasks/comms_manager.c: error: sim_beacon_count undeclared

src/tasks/telemetry_task.c: error: sim_magnetic_B undeclared ... (same error

repeated across all 7 task files)
```

Root cause: The variables sim_angular_rate, sim_pointing_err, sim_magnetic_B, and sim_beacon_count were declared as extern in sim_hal.h but the declarations were appended AFTER the #endif guard. The compiler never saw them because they were outside the #ifndef SIM_HAL_H block. Everything inside #ifndef/#endif is compiled; everything after #endif is ignored.

What was wrong:

```
/* BROKEN sim_hal.h — declarations after #endif */

#ifndef SIM_HAL_H

#define SIM_HAL_H

/* ... all existing declarations ... */ #endif /*

SIM_HAL_H <-- compiler stops reading here */


extern float sim_angular_rate; /* INVISIBLE -- after #endif! */

extern float sim_pointing_err; /* INVISIBLE -- after #endif! */
```

Fix — rewrite sim_hal.h with ALL externs inside the guard:

```
/* FIXED sim_hal.h -- ALL externs inside the guard */

#ifndef SIM_HAL_H

#define SIM_HAL_H

#include <stdint.h>

#include <stdbool.h>


typedef enum { MODE_SAFE=0, MODE_NOMINAL=1, MODE_SCIENCE=2 } SystemMode;

extern SystemMode sim_mode;

extern float sim_soc;

extern float sim_battery_v;

extern uint8_t sim_max_passes;

extern float sim_att_err;

extern uint8_t sim_adcs_mode;

extern float sim_mtq_dipole[3];
extern float sim_angular_rate; /* NOW VISIBLE */

extern float sim_pointing_err; /* NOW VISIBLE */

extern float sim_magnetic_B[3]; /* NOW VISIBLE */

extern uint32_t sim_beacon_count; /* NOW VISIBLE */ /* ... all other externs ...

*/

void sim_reset_state(void);


#endif /* SIM_HAL_H */
```

Apply the fix:

```
cat > src/hal/sim_hal.h << EOF /* paste the complete fixed

sim_hal.h above */

EOF
```

Error 2 — usleep implicit declaration

Error message:

```
src/main.c: error: implicit declaration of function usleep;

did you mean sleep? [-Wimplicit-function-declaration]
```

Root cause: The -std=c11 flag strictly follows the C11 standard, which does not include POSIX extensions. The usleep() function is part of POSIX, not ISO C. With -std=c11 the compiler does not expose usleep() even if unistd.h is included. Switching to -std=gnu11 enables GNU extensions which include all POSIX functions.

Fix — change -std=c11 to -std=gnu11 in Makefile:

```
# BROKEN Makefile CFLAGS = -Wall -Wextra -Wno-unused-parameter -
std=c11 -O0 -g -I./src ^^^^^^^^
strict C11 -- no POSIX

# FIXED Makefile CFLAGS = -Wall -Wextra -Wno-unused-parameter -
std=gnu11 -O0 -g -I./src
^^^^^^^^^
GNU C11 -- includes POSIX
```

Apply the fix:

```
sed -i 's/-std=c11/-std=gnu11/' Makefile # Or
open with nano and change the line manually
```

Error 3 — sim_state.c had duplicate / appended definitions

Error message:

```
# When building test_tc or test_tm after adding task support:
src/hal/sim_state.c: error: redefinition of sim_angular_rate
src/hal/sim_state.c: error: redefinition of sim_beacon_count
```

Root cause: When the extra declarations were appended to sim_hal.h and sim_state.c during development, some variables got defined twice in sim_state.c. C does not allow two global definitions of the same variable. The fix was to rewrite sim_state.c from scratch as a single clean file with every variable defined exactly once.

What was wrong:

```
/* BROKEN sim_state.c -- duplicate definitions */
float sim_soc = 0.80f; /* defined here
*/ /* ... many lines ... */ /* appended
section: */
float sim_angular_rate = 5.2f; /* defined again -- ERROR! */
uint32_t sim_beacon_count = 0; /* defined again -- ERROR! */
```

Fix -- rewrite sim_state.c as single clean file:

```
cat > src/hal/sim_state.c << EOF

#include "sim_hal.h"

#include <string.h> SystemMode

sim_mode = MODE_NOMINAL;

float sim_soc = 0.80f;

float sim_battery_v = 7.4f;

uint8_t sim_max_passes = 3;

float sim_att_err = 2.0f;

uint8_t sim_adcs_mode = 2;

float sim_mtq_dipole[3] = {0.0f, 0.0f, 0.0f};

float sim_angular_rate = 5.2f; /* defined ONCE */

float sim_pointing_err = 45.0f; /* defined ONCE */

float sim_magnetic_B[3] = {25e-6f, 5e-6f, -40e-6f};

uint32_t sim_beacon_count = 0; /* defined ONCE */

/* ... all other definitions ... */

void sim_reset_state(void) { /* ... */ }

EOF
```

16. Build and Run — Complete Steps

Commands in order:

```
cd ~/FreeRTOS-Kernel/SLARS-BCG-1-OBSW

# Apply sim_hal.h fix (rewrite with all externs inside guard)
cat > src/hal/sim_hal.h << EOF
# ... paste fixed sim_hal.h
...
EOF

# Apply sim_state.c fix (rewrite as single clean file)
cat > src/hal/sim_state.c << EOF
# ... paste fixed sim_state.c
...
EOF

# Apply Makefile fix
sed -i 's/-std=c11/-std=gnu11/' Makefile

# Build everything
make clean
make all

# Run Step A and Step B tests
./test_tc
./test_tm

# Run the simulation (Ctrl+C to stop)
./slars_obsw_sim
```

17. Expected Simulation Output

```
****************************************************
* SLARS-BCG-1 FLIGHT SOFTWARE v1.0 *
* Bandaranayake College, Gampaha, Sri Lanka *
* ORBIT: 500 km LEO | 10 deg | 5-6 passes/day *
****************************************************
[BOOT] Stage 1: Hardware init...
[HAL] Simulation HAL initialized [BOOT] Stage 2: Initialising 7 tasks (MTQ-Only)...
[BOOT] Watchdog P8 100ms
[BOOT] ADCS P7 100ms
```

```
[BOOT] FDIR Monitor P6 2000ms
[BOOT] Power Mgr P6 1000ms
[BOOT] COMMS Mgr P5 500ms
[BOOT] Payload Sched P5 1000ms
[BOOT] Telemetry P4 10000ms
[BOOT] Flash Mgr P3 5000ms
[BOOT] NOTE: No RW monitor task (MTQ-Only)
[BOOT] Stage 3: Subsystem health check...
[BOOT] EPS: soc=80% vbat=7.9V
[BOOT] ADCS: rate=5.2deg/s mode=2 (B-dot) [BOOT] Stage 5:
Mode determination... Starting mode: NOMINAL
[BOOT] Stage 6: SLARS-BCG-1 OPERATIONAL
[WDT] Watchdog started P8 100ms [WDT] Monitoring 7 tasks
(MTQ-Only, no RW monitor). [COMMS] Beacon #1 TX --
soc=80% mode=1 adcs=2 att=43.9deg
[FDIR] Health OK: soc=80% rate=5.2 att=40.7deg
```

Press Ctrl+C to stop the simulation.

OBSW SIMULATION RUNNING | 7 FreeRTOS Tasks | Steps A-C Complete | 104/104 Tests Passing |